

Contents

1 To-Do Class	1
1.0.1 Explanation:	2
1.0.2 Usage Example:	2
1.0.3 Security Considerations:	3

1 To-Do Class

Let's create a comprehensive `Todo` class for our To-Do application in PHP. This class will include methods for various CRUD operations such as creating, reading, updating, and deleting To-Do items, as well as marking them as done or undone. Here's how you can structure it:

```
<?=  
class Todo {  
    private $pdo;  
  
    public function __construct($pdo) {  
        $this->pdo = $pdo;  
    }  
  
    public function create($userId, $title, $description, $categoryId) {  
        $sql = "INSERT INTO todos (user_id, title, description, category_id) VA  
        $stmt = $this->pdo->prepare($sql);  
        $stmt->execute([$userId, $title, $description, $categoryId]);  
    }  
  
    public function read($todoId) {  
        $sql = "SELECT * FROM todos WHERE id = ?";  
        $stmt = $this->pdo->prepare($sql);  
        $stmt->execute([$todoId]);  
        return $stmt->fetch();  
    }  
  
    public function update($todoId, $title, $description, $categoryId) {  
        $sql = "UPDATE todos SET title = ?, description = ?, category_id = ? WH  
        $stmt = $this->pdo->prepare($sql);  
        $stmt->execute([$title, $description, $categoryId, $todoId]);  
    }  
  
    public function delete($todoId) {  
        $sql = "DELETE FROM todos WHERE id = ?";  
        $stmt = $this->pdo->prepare($sql);  
        $stmt->execute([$todoId]);  
    }  
}
```

```

    public function markDone($todoId) {
        $sql = "UPDATE todos SET done = TRUE WHERE id = ?";
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute([$todoId]);
    }

    public function markUndone($todoId) {
        $sql = "UPDATE todos SET done = FALSE WHERE id = ?";
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute([$todoId]);
    }

    public function getAllTodosByUser($userId) {
        $sql = "SELECT * FROM todos WHERE user_id = ?";
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute([$userId]);
        return $stmt->fetchAll();
    }
}

```

1.0.1 Explanation:

- **Constructor:** The constructor takes a PDO database connection object. This object is used for all database operations within the class.
- **Create Todo:** The `create` method adds a new todo item to the database for a specific user.
- **Read Todo:** The `read` method fetches the details of a specific todo item by its ID.
- **Update Todo:** The `update` method modifies the details of an existing todo item.
- **Delete Todo:** The `delete` method removes a todo item from the database.
- **Mark Done/Undone:** The `markDone` and `markUndone` methods update the status of a todo item to either completed or not completed.
- **Get All Todos:** The `getAllTodosByUser` method retrieves all todo items for a specific user.

1.0.2 Usage Example:

Here's an example of how you might use this class in your application:

```

<?php
$pdo = new PDO("mysql:host=localhost;dbname=your_database", "username", "password");
$todo = new Todo($pdo);

// Create a new Todo
$todo->create(1, "Buy groceries", "Milk, eggs, bread", 2);

```

```
// Update a Todo
$todo->update(1, "Buy groceries and vegetables", "Milk, eggs, bread, and carrots")

// Mark a Todo as done
$todo->markDone(1);

// Get all Todos for a user
$userTodos = $todo->getAllTodosByUser(1);
```

1.0.3 Security Considerations:

- Always use prepared statements as done in the methods to prevent SQL Injection attacks.
- Validate and sanitize all inputs before passing them to these methods.
- Handle exceptions and errors appropriately, especially for database operations.
- Ensure that the user performing the operations has the right permissions, especially for updating and deleting operations.

This `Todo` class provides a solid foundation for managing todos in our application with standard CRUD operations and specific functionalities like marking todos as done or undone.